Інструкція до практичної роботи №99
ТЕМА: "Створення власного проекту."

Мета роботи. Навчитись використовувати графіку в середовищі C++ Builder.

Теоретичні відомості

C++ Builder надає зручні та прості у застосуванні засоби для роботи з графічними файлами растрових зображень (бітових образів у форматі *.bmp, піктограм *.ico чи метафайлів *.wmf (*.emf)). Для цього визначено похідні від базового класу TGraphic, який надає мінімальний стандартний інтерфейс для роботи з графікою, об’єктні класи TBitmap, TIcon та TMetafile. З цих класів клас TBitmap є єдиним графічним класом з канвою.

Канва (властивість Canvas) графічних компонентів дозволяє програмістові при роботі з графікою не вникати в проблеми стеження за системними ресурсами, а лише визначити властивості та характеристики графічних інструментів для використовуваних компонентів.

Зазвичай робота з зображенням провадиться лише з частиною, яка відкрита для доступу через контейнерний клас TPicture. Він є інкапсульований у клас TImage і візуальний компонент TImage, як їхній основний компонент та має можливість роботи з іконками, bmp-файлами і метафайлами. А при долученні заголовного файла jpeg.hpp можна опрацьовувати файли форматів *.jpeg, *.jpg.

Методи LoadFromFile() та SaveToFile()дозволяють динамічно завантажувати та зберігати файли зображень форматів bmp та ico до компонентів типу TImage:

```
Image1->Picture->LoadFromFile("1.bmp");
Image1->Picture->SaveToFile("2.bmp");
```

або

```
Image1->Picture->LoadFromFile("1.ico");
Image1->Picture->SaveToFile("2.ico");
```

Для того, щоби звертатися до метафайлів формату emf (оновлена версія формату метафайлів фірми Microsoft), слід скористатися властивістю Enhanced, встановивши її в значення true. Значення false властивості Enhanced використовується для більш старої версії метафайлів wmf.

```
Image1->Picture->Metafile->Enhanced=true;
Image1->Picture->LoadFromFile("1.emf");
Image1->Picture->SaveToFile("2.emf");
```

Що стосується візуального компонента TImage, то він призначений для розміщення на формі графічного зображення (бітового образу, піктограми чи метафайла). Більшість властивостей цього компонента є такі самі, як і багатьох інших компонентів (розташування зображення на формі, його розміри тощо). Зосередимо увагу на “нестандартних” властивостях цього компонента:

Picture – контейнер для завантаження файлів зображень чи то при створенні форми чи програмно за допомогою методів LoadFromFile() та SaveToFile();

AutoSize – значення true цієї властивості автоматично змінює (збільшує чи зменшує) розміри контейнера до розмірів зображення;

Stretch – значення true цієї властивості вписує (розтягує чи стискає) зображення у рамки компонента, але оскільки навряд чи реально встановити розміри Image точно

пропорційними розміру малюнка, то зображення спотвориться. Тому встановлювати Stretch у true доцільно лише для візерунків, але не для картинок. Властивість Stretch не діє на зображення піктограм ico, які не можуть змінювати свої розміри;

Center – вказує, чи потрібно центрувати зображення у межах компонента. Ігнорується, якщо Autosize = true або якщо Stretch = true і зображення не є піктограмою (ico);

IncrementalDisplay – якщо true, то при завантаженні великих файлів вони будуть відбиватися частинами по мірі завантаження;

Transparent – прозорість фону, яку можна використовувати для накладення зображень одне на одне, щоби верхня картинка не затуляла нижню. Одне з можливих застосувань цієї властивості – накладення на картинку написів, виконаних у вигляді бітової матриці. Кольором прозорості береться колір лівого нижнього кутового пікселя.

Canvas – програмно доступна властивість, яка містить канву для виведення растрового зображення (але не піктограми чи метафайла!).

Для доступу до графічних файлів у C++ Builder існують спеціальні компоненти на закладці Dialogs:

OpenPictureDialog

– відкрити рисунок. Реалізує спеціальне вікно вибору графічних файлів з можливістю попереднього переглядання рисунків;

SavePictureDialog

– зберегти рисунок. Реалізує спеціальне вікно зберігання графічних файлів з можливістю попереднього переглядання рисунків.

Усі властивості цих компонентів однакові, тільки їх сенс дещо різний для відкриття і закриття файлів:

FileName – основна властивість, в якій повертається у вигляді рядка обраний користувачем файл. Значення цієї властивості можна задати і перед зверненням до діалогу;

Filter – фільтр, який задає один, декілька чи усі доступні типи графічних файлів зі списку: *.jpg; *.jpeg; *.bmp; *.ico; *.emf; *.wmf. Один від одного шаблони відокремлюються вертикальними рисками;

FilterIndex – визначає номер фільтра, який буде за замовчуванням показаний користувачеві у момент відкриття діалогу. Наприклад, значення FilterIndex = 1 задає за замовчуванням перший фільтр;

InitialDir – визначає початковий каталог, який відкриється у момент початку роботи користувача з діалогом. Якщо значення цієї властивості не задане, то відкриється поточний каталог чи той, який був відкритий при останньому зверненні користувача до відповідного діалогу;

DefaultExt – визначає значення розширення файла за замовчуванням. Якщо значення цієї властивості не задане, користувач повинен вказати у діалозі повне ім’я файла з розширенням. Якщо ж задати значення DefaultExt, то користувач може писати у діалозі ім’я без розширення. У цьому разі буде прийнято задане розширення;

Title – дозволяє задавати заголовок діалогового вікна. Якщо цю властивість не задано, вікно відкривається із заголовком, визначеним операційною системою;

Options – визначає умови вибору файла. Множина опцій, які можна встановлювати програмно чи під час проектування, основні з яких:

- ofAllowMultiSelect – дозволяє користувачеві вибирати декілька файлів;
- ofCreatePrompt – у разі, якщо користувач написав ім'я неіснуючого файла, з'явиться зауваження і запит, чи треба створити файл із заданим ім'ям;
- ofEnableIncludeNotify – дозволяє посилати в діалог повідомлення;
- ofEnableSizing – дозволяє користувачеві змінювати розмір діалогового вікна;
- ofFileMustExist – у разі, якщо користувач написав ім'я неіснуючого файла, з'явиться повідомлення про помилку;
- ofNoChangeDir – після клацання користувача на кнопці "OK" відновлює поточний каталог, незалежно від того, який каталог був відкритий при пошуку файла;
- ofPathMustExist – генерує повідомлення про помилку, якщо користувач вказав в імені файла неіснуючий каталог;
  - ofOverwritePrompt – у разі, якщо при збереженні файла користувач написав ім'я існуючого файла, з'явиться зауваження, що файл з таким ім'ям існує, і запит до користувача переписати існуючий файл;
  - ofHideReadOnly – видаляє з діалогу індикатор "Відкрити тільки для читання";
  - ofReadOnly – за замовчуванням встановлює індикатор "Відкрити тільки для читання" при відкритті діалогу.

За замовчуванням усі опції, окрім ofHideReadOnly, є вимкнені. Проте, як видно з їхнього опису, більшість з них доцільно встановлювати перед викликом діалогів.

Якщо ви дозволяєте за допомогою опції ofAllowMultiSelect множинний вибір файлів, то список обраних файлів можна прочитати у властивості Files типу TStrings.

Проілюструємо на прикладі роботу з компонентами Image, OpenPicture-Dialog, SavePictureDialog та методами LoadFromFile() та SaveToFile().

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Тема й обсяг альбому залежать від N (mod — остача від ділення):

Дані	**Правило**	**Приклад N = 7**
Тема альбому	N mod 3 = 0, 1, 2: Природа, Місто, Спорт	Місто
Обсяг	не менше 5 + N mod 3 фотографій	не менше 6

б) Здайте архівом папку проєкту Прізвище_ПР99 (файли *.bpr, *.cpp, *.h, *.dfm, *.res та файли даних, потрібні для запуску (зображення, звуки)) і документ Word зі звітом; виконуваний файл *.exe додавати не потрібно.

в) До звіту додайте скріншоти форми в режимі проєктування й під час роботи програми (з введеними даними) та текст змінених вами обробників подій.
г) Заголовок форми — «Фотоальбом: тема»; кнопки «Перейти до наступного фото», «Видалити фото з альбому», «Змінити фото на інше», «Долучити нове фото до альбому» (за потреби п'ята кнопка за умовою); список файлів зберігається в photo.dat, при запуску альбом завантажується автоматично; підключіть jpeg.hpp для JPEG-файлів.
д) У коді залиште короткі коментарі власними словами перед основними блоками.
е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

Хід роботи

Завдання. Написати програму-фотоальбом, яка дозволяє переглядати файлів зображень, наприклад фотографій, і редагувати (долучати, вилучати, змінювати) вміст альбому.

> **Приклад (N = 7):** тема «Місто», 6 фото; при запуску показується перше фото, у StatusBar — «1 файл зі списку» та шлях до файлу; «Видалити фото» просить підтвердження (OK/Cancel); при порожньому альбомі — повідомлення «Альбом порожній. Скористайтесь відповідною кнопкою для долучення фото до альбому».

На формі розмітимо компонент ListBox1 для переглядання списку фотографій альбому та компонент Image1 для посереднього переглядання самих фото. Щоб невеликі фотографії виводились посередині компонента Image1 встановимо властивість Center у значення true. Напроти для виведення усього, а не лише фрагмента, зображення великих фото встановимо властивість AutoSize у значення true. Крім того, використаємо спеціалізований діалог для відкриття графічних файлів OpenPictureDialog (закладка Dialogs) та компонент StatusBar (закладка Win32) для виведення панелі у нижній частині форми з інформацією про ім’я та місцезнаходження файла, який переглядається.

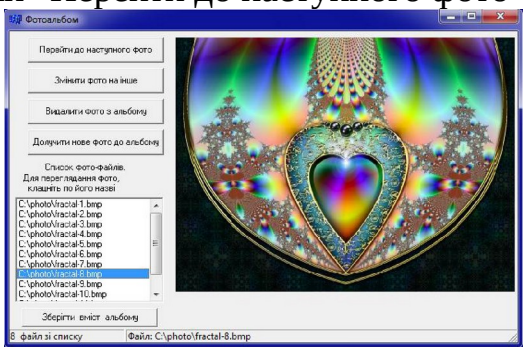
Взагалі кажучи, компонент StatusBar доцільно використовувати для виведення різноманітної службової інформації, яка дозволяє вести візуальний контроль за діями програмного додатку. Щоби створити багатосекційну панель (у нашому прикладі – двосекційну) на етапі створення форми, слід двічі клацнути по компонентові на формі та у віконці редагування панелей, яке відкриється, скористатись іконкою “Add new Ins” для створення бажаної кількості секцій панелі. Нумерація індексів секцій розпочинається з нуля, а розмір ширини кожної секції можна змінити зі значення у 50 пікселів за замовчуванням, приміром на 150, у вікні інспектора об’єктів (Object Inspector), попередньо виокремивши мишкою певну секцію. Аналогічні за дією команди можна прописати у програмі, як приміром у наведеному прикладі для функції **FormCreate**.

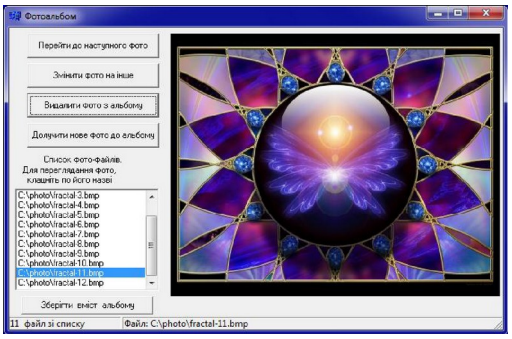
Крім того зауважимо, що за замовчуванням компоненти Image та OpenPictureDialog забезпечують доступ лише до файлів зображень у форматах BMP (бітовий образ), ICO (піктограму), WMF чи EMF (метафайли). Щоби забезпечити

доступ і до графічних файлів у форматах JPEG та JPG, залучимо до програми бібліотеку **jpeg.hpp**.

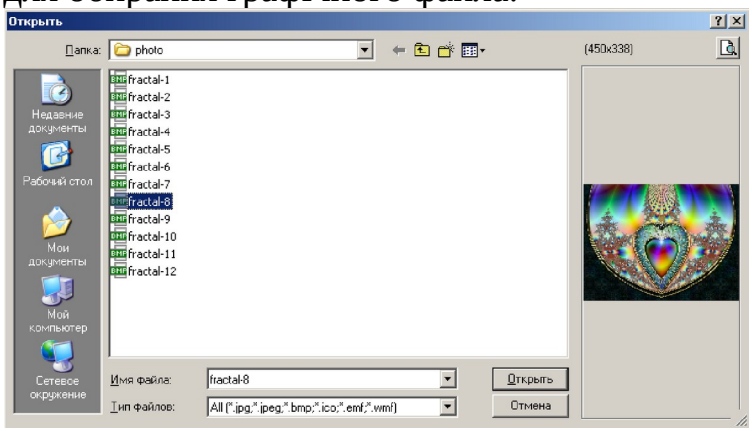
Відмінності компонента OpenPictureDialog від подібного компонента OpenDialog полягають у наявності фільтрів для вибирання лише графічних файлів з відповідними розширеннями та наявності у діалоговому вікні додаткової панелі попереднього переглядання обраного файла.

Вікно форми проекту з результатами роботи програми за натискання на командні кнопки “Перейти до наступного фото” та “Видалити фото з альбому” матиме вигляд:





При клацання на кнопки “Змінити фото на інше” та “Долучити нове фото до альбому” за допомогою компонента OpenPictureDialog1 відкриватиметься діалогове вікно для обирання графічного файла:



Текст програми:

```
#include "jpeg.hpp" // Забезпечує роботу з графічними JPEG-файлами
AnsiString albom="photo.dat"; // Ім‘я файла зі списком файлів альбому
//---------------------------------------------------------
// Клацання по обраному фото у списку
void __fastcall TForm1::ListBox1Click(TObject *Sender)
{ Image1->Picture->LoadFromFile(ListBox1->Items->
Strings[ListBox1->ItemIndex]);
StatusBar1->Panels->Items[0]->Text=IntToStr(ListBox1->
ItemIndex+1)+" файл зі списку";
StatusBar1->Panels->Items[1]->Text="Файл: "+ListBox1->Items->
Strings[ListBox1->ItemIndex];
}
//---------------------------------------------------------
```

```
void __fastcall TForm1::FormCreate(TObject *Sender)
{Image1->AutoSize=false; // Заборонити змінювати розміри
// компонента залежно від розмірів зображення
Image1->Proportional=true; // Зберігати пропорції зображень
Image1->Stretch=0; // Заборонити "підганяти" розміри
// зображення до розмірів компонента
StatusBar1->Panels->Add(); // Створити секцію на панелі (індекс – 0)
StatusBar1->Panels->Items[0]->Width=150; // Нова ширина першої секції
StatusBar1->Panels->Add(); // Створити ще одну секцію на панелі (індекс – 1)
if(FileExists(albom)) // Якщо альбом існує,
{ ListBox1->Items->LoadFromFile(albom); // завантажити список файлів
if(ListBox1->Items->Count>0) // та, якщо список не є пустий,
{ ListBox1->ItemIndex=0; // виокремити саме перше у списку фото
ListBox1Click(NULL); // та показати його за допомогою
} // команд функції ListBox1Click()
}
else ShowMessage("Альбом порожній. Скористайтесь відовідною кнопкою для долучення фото до альбому");
}
//---------------------------------------------------------
// Перейти до наступного фото
void __fastcall TForm1::Button1Click(TObject *Sender)
{ ListBox1->ItemIndex++;
ListBox1Click(NULL);
}
//---------------------------------------------------------
// Змінити фото на інше
void __fastcall TForm1::Button2Click(TObject *Sender)
{ if(OpenPictureDialog1->Execute())
{ // Обрати нове фото і вставити його на місце небажаного фото
Image1->Picture->LoadFromFile(OpenPictureDialog1->FileName);
ListBox1->Items->Insert(ListBox1->ItemIndex,
OpenPictureDialog1->FileName);
// Видалити небажане фото
ListBox1->Items->Delete(ListBox1->ItemIndex);
} }
//---------------------------------------------------------
// Видалити фото з альбому
void __fastcall TForm1::Button3Click(TObject *Sender)
{ int btn=MessageDlg("Ви дійсно бажаєте видалити обране фото
з альбому?", mtConfirmation,mbOKCancel,0);
if(btn = = 1) // Якщо обрано кнопку OK, видалити фото
{ ListBox1->Items->Delete(ListBox1->ItemIndex); // зі списку
if(ListBox1->Items->Count>0) // та, якщо інші фото залишились,
```

```
{ ListBox1->ItemIndex=0; // виокремити саме перше у списку фото
ListBox1Click(NULL); // та показати його за допомогою
} } } // команд функції ListBox1Click()
//---------------------------------------------------------
// Долучити нове фото до альбому
void __fastcall TForm1::Button4Click(TObject *Sender)
{ if(OpenPictureDialog1->Execute())
{ Image1->Picture->LoadFromFile(OpenPictureDialog1->FileName);
ListBox1->Items->Insert(ListBox1->ItemIndex+1,
OpenPictureDialog1->FileName);
ListBox1->ItemIndex++;
ListBox1Click(NULL);
} }
//---------------------------------------------------------
// Зберігти вміст альбому
void __fastcall TForm1::Button5Click(TObject *Sender)
{ ListBox1->Items->SaveToFile("photo.txt"); // Зберегти список фото
}
```

Критерії оцінювання

Складова	**Оцінювання**	**Макс. балів**
Форма й компоненти (ListBox, Image, OpenPictureDialog, StatusBar)	3 бали	3
Перегляд і видалення фото	2 бали	2
Заміна й додавання фото	3 бали	3
Збереження й завантаження альбому (photo.dat)	2 бали	2
Охайність інтерфейсу, фото за темою	1 бал	1
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (не виконано вимогу умови, альбом не зберігається, не оброблено порожній альбом, немає підтвердження видалення) — 50%; програма не працює або не здана — 0. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити тему альбому чи підпис кнопки). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

1. Для чого призначені компоненти ListBox і OpenPictureDialog?
2. Як зберегти список файлів альбому у текстовий файл і завантажити його при запуску?

3. Як вивести службову інформацію в StatusBar?
4. Як забезпечити відображення JPEG-файлів у компоненті Image?

Відповідайте на контрольні питання письмово у звіті (документ Word разом зі скріншотами): 1–2 речення на кожне питання, власними словами.